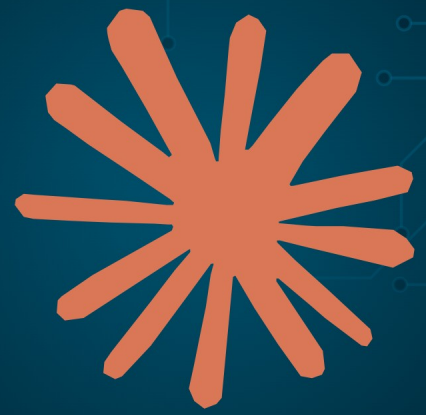
Claude ANTHROP\C

PROGRAMA DE CAPACITACIÓN

Desarrollando con IA

Unidad 3 · Generar, refactorizar y documentar

Día 2 · Bloque mañana · Instructor: Felipe Cuevas



felipecuevas.dev

DÍA 2 · RECORRIDO

¿Dónde estamos?

U1 ✓ U2 ✓ **U3 · hoy** U4 U5 U6 U7 U8 U9 U10

NIVEL DE AUTONOMÍA

20 → 30

De guiar con una spec a construir, refactorizar y documentar.



CASO HILO · PEDIDOS-WEB

Sprint 2

Ya tienes la API de pedidos de la U2. Hoy: el detalle del pedido y un módulo heredado.

UNIDAD 3 · HOY

Cuatro ideas para esta mañana

C1

Pasos chicos

Una tarea, un incremento, un commit.

C2

Red de tests

Sin tests no se refactoriza.

C3

Skills

Enséñale una vez, lo usa siempre.

C4

Desconfía y verifica

Lee, ejecuta y pide que explique.

Material oficial: *Best practices* · *Common workflows* · *Introduction to agent skills* · *AI Fluency for builders*

CONCEPTO 1 · 15 MIN

Pasos chicos

01

Una tarea, un incremento, un commit.

Generación incremental | Contexto preciso | Diffs revisables

C1 · PASOS CHICOS

Un diff gigante no se revisa, se aprueba

Un pedido gigante

1 commit



~600 líneas mezcladas: modelo, lógica y tests

⚠ Si algo falla, ¿qué parte revertimos?

Tres incrementos

3 commits



~80 líneas cada uno, con tests en verde

✓ Cada paso se lee, se prueba y se revierte solo.

Si no lo revisas en 5 minutos, el paso es demasiado grande.

C1 · PASOS CHICOS

Pide poco, pero con contexto preciso

Menciona el archivo

`@src/TodoMini/TareaService.cs`

Pega el error

El stack trace completo, no «no funciona».

Adjunta el mockup

Una imagen dice más que tres párrafos.

Convenciones fijas

Scaffolding y estilo en AGENTS.md.

```
> Solo el paso 1: agrega a Tarea una lista de etiquetas con sus tests.
  No toques TareaService todavía. Corre dotnet test al terminar.
```

DEMO EN VIVO · 6 MIN · TODOMINI

El commit gigante vs. tres commits chicos

A · Un solo pedido

```
> Agrega etiquetas a las tareas: modelo,
etiquetar, filtrar y tests.
```

Un diff largo, un commit.

B · Tres incrementos

1. Etiquetas en Tarea + tests ✓
2. TareaService.Etiquetar(id, etiqueta) + tests ✓
3. TareaService.ListarPorEtiqueta(etiqueta) + tests ✓

Commit después de cada paso en verde.

Votación: ¿cuál de los dos diffs revisarías en 5 minutos?

TODOMINI · 1 · Etiquetas → 2 · Refactor → 3 · Skill

CONCEPTO 2 · 18 MIN

Red de tests

02

Sin tests no se refactoriza.

Refactorizar | Ciclo seguro | /rewind y git

Refactorizar: cambia la forma, no el comportamiento

Es refactor

Renombrar proc2() a algo que se entienda

Extraer un método de un bloque largo

Eliminar código duplicado

No es refactor

Corregir un bug «de paso»

Agregar una validación nueva

Cambiar el formato de la respuesta

Los mismos tests deben pasar antes y después.

C2 · RED DE TESTS

El ciclo seguro

1

Tests en verde

Si falta uno, escríbelo antes.

2

Un cambio chico

Renombrar, extraer, deduplicar.

3

Ejecutar tests

`dotnet test`

Verde → commit

Y vuelves al paso 2.

Rojo → volver atrás

Con git o con /rewind. No «arreglar encima».

C2 · RED DE TESTS

Volver atrás: /rewind y git

	/rewind (checkpoints)	git
Qué guarda	Un punto antes de cada prompt	Los commits que tú decides
Cómo vuelves	Esc Esc o /rewind	git restore · git reset
Restaura	Código, conversación o ambos	Código, con historial permanente
No cubre	Archivos que solo cambió Bash y cambios externos	Lo que no commiteaste

/rewind para el minuto a minuto. git para lo que no puedes perder.

Fuente: Claude Code Docs · Checkpointing (consulta 2-nov-2026)

DEMO EN VIVO · 6 MIN · TODOMINI

Refactor con red



```
public string ResumenTexto() {
  var t = _tareas.Count;
  int x = 0, c = 0;
  foreach (var a in _tareas)
    if (a.Completada) x++;
  foreach (var b in _tareas) // otra vez
    if (!b.Completada) c++;
  return $"{t} tareas: {x} completada{(x == 1 ? "" : "s")}, "
       + $"{c} pendiente{(c == 1 ? "" : "s")}";
}
```

1. Claude escribe primero los tests
2. Refactoriza en pasos chicos, con dotnet test en cada paso
3. **Metemos un error a propósito: ¿lo atrapa un test?**

> Ya agregamos etiquetas; ahora ordenamos un método viejo del mismo proyecto.

TODOMINI · 1 · Etiquetas → 2 · Refactor → 3 · Skill

CONCEPTO 3 · 25 MIN

Skills

03

Enséñale una vez, lo usa siempre.

SKILL.md | Activación | Skill o AGENTS.md | planificador-proyectos-web

Anatomía de una Skill

Una carpeta

```
.claude/skills/
└ escribir-readme/
   ├ SKILL.md
   ├ references/
   ├ scripts/
   └ assets/
```

Solo SKILL.md es obligatorio.

</> SKILL.md

```
---
name: escribir-readme
description: Escribe o actualiza el README
  de un proyecto. Úsala cuando pidan documentarlo…
---
# Pasos
1. Lee el código antes de escribir…
```

Frontmatter (qué y cuándo) + instrucciones (cómo).

Fuente: Claude Code Docs · Agent Skills (consulta 2-nov-2026)

Cómo se activa: divulgación progresiva

Siempre	**name + description de cada Skill**
Al activarse	El cuerpo de SKILL.md
Si hace falta	references/ y scripts/

Menos contexto gastado, más Skills disponibles.

Automática

Tu pedido calza con la description.

Manual

`/escribir-readme`

Fuente: Claude Code Docs · Agent Skills (consulta 2-nov-2026)

¿Skill o AGENTS.md?

`AGENTS.md`

Reglas que aplican siempre

«Usamos xUnit y nombres en español.»

«Nunca hagas commit por mí.»

`SKILL.md`

Un procedimiento, cuando corresponde

«Cómo documentamos un módulo.»

«Cómo planificamos un proyecto.»

Estándar abierto. Agent Skills (SKILL.md) también lo adoptan otras herramientas, como Codex, GitHub Copilot y Gemini CLI. Lo propio de Claude Code son campos extra del frontmatter.

La description lo es todo

⚠ `description: Ayuda con documentación.` — Nunca se activa

✓ `description: Escribe o actualiza el README.md de un proyecto: propósito, instalación, uso y estructura. Úsala cuando pidan documentar el proyecto o mejorar el README, aunque no nombren la Skill.`

1. Qué hace
2. Cuándo usarla
3. Lo clave, primero

Fuente: Claude Code Docs · Agent Skills (consulta 2-nov-2026)

DEMO EN VIVO · 6 MIN · TODOMINI

Mi primera Skill en vivo

```
$ mkdir -p .claude/skills/escribir-readme
# SKILL.md: name + description + pasos
$ claude
> Hazme un README para este proyecto.
● Skill(escribir-readme)
```

¿Se activó sin nombrarla? Si no, corrige la description.

✓ ¿Siguió los pasos de la Skill?

¿El README ya incluye las etiquetas y el ResumenTexto ordenado?

Agregamos y ordenamos; ahora enseñamos a Claude a documentarlo.

TODOMINI 1 · Etiquetas → 2 · Refactor → 3 · Skill

★ Una Skill real · hecha por el instructor

Licencia MIT

planificador-proyectos-web

Planifica y documenta un proyecto web antes de que el agente programe.
SDD más prácticas de Scrum, sin ceremonias.



github.com/ffelipecuevasc/planificador-proyectos-web

★ PLANIFICADOR-PROYECTOS-WEB

Anatomía de una Skill profesional

`SKILL.md · 154 líneas`	Instrucciones breves (entre ellas, no hacer commits); description rica en «cuándo usarla»
`references/ · 4 guías`	Entrevista, niveles, tipos: se leen solo si hacen falta
`assets/plantillas/`	AGENTS.md, DESIGN.md, specs, backlog y OpenAPI
`scripts/validar_documentacion.py`	Valida antes de entregar: sin criterios vagos ni datos inventados
`genera .claude/settings.json`	Bloquea editar los documentos inmutables y, en APIs con base de datos, los comandos contra bases remotas

Documenta según la complejidad: **Pequeño** **Mediano** **Complejo**

Fuente: github.com/ffelipecuevasc/planificador-proyectos-web

★ PLANIFICADOR-PROYECTOS-WEB

Instálala y úsala

Claude Code

```
git clone …/planificador-proyectos-web.git
cp -r planificador-proyectos-web/skills/
  planificador-proyectos-web
  ~/.claude/skills/
```

O en .claude/skills/ de un solo proyecto. Windows: Copy-Item.

Claude (web y app)

Descarga el .zip de Releases

Customize → Skills → Upload

Requiere la ejecución de código activada.

▷ /planificador-proyectos-web

⚠ **Copia la subcarpeta skills/…, no la raíz del repo.**

Fuente: README del repositorio (requisitos verificados por el autor en oct-2026)

CONCEPTO 4 · 12 MIN

Desconfía y verifica

04

Lee, ejecuta y pide que explique.

Discernimiento | Documentación verificada | Modelo según la tarea

C4 · DESCONFÍA Y VERIFICA

Tres verbos antes de aceptar

Leer

Todo el diff, no solo el resumen del agente.

Ejecutar

Tests, la app y el caso que te preocupa.

Preguntar

«¿Por qué elegiste esto?»

«¿Qué caso borde no cubriste?»

La documentación generada también se verifica: **README, ADR y diagramas Mermaid** contra el código.

C4 · DESCONFÍA Y VERIFICA

El modelo se elige según la tarea

`/model`

TAREA ACOTADA Y CLARA

Modelo rápido

`sonnet · haiku`

Renombrar, un test, un ajuste chico.

CAMBIO AMPLIO O AMBIGUO

Modelo más capaz

`opus`

Varios archivos, diseño o un bug difícil.

SIEMPRE

Considera cuota y tiempo

`/usage`

Más capaz suele ser más lento y gasta más cuota del asiento.

Cómo cambiar: /model abre el selector · /model sonnet cambia directo.

Para practicar en casa: mismo pedido, dos modelos. Cuida la cuota de tu asiento.

Fuente: Claude Code Docs · Model configuration (consulta 9-nov-2026)

REPASO · 5 MIN

Antes del receso: cuatro preguntas



C1 PASOS CHICOS

¿Cómo sabes que un paso es lo bastante chico?



C2 RED DE TESTS

¿Qué debe existir antes de tocar código que no escribiste?



C3 SKILLS

¿Qué decide si Claude usa tu Skill sin que la nombres?



C4 DESCONFÍA Y VERIFICA

¿Qué haces antes de aceptar lo que hizo Claude?

PARA INVESTIGAR POR TU CUENTA

Para profundizar

Refactor arquitectónico

Separación de capas e inversión de dependencias.

Skills avanzadas

allowed-tools, divulgación progresiva y scripts.

Comandos y estilos

Comandos personalizados y estilos de salida.

Probar interfaces

Pruebas de UI web con Claude in Chrome.

Cursos y guías

Introduction to agent skills y docs de Skills.

★ planificador-proyectos-web

Instálalo y úsalo en un proyecto personal.

15'

Receso

A la vuelta: taller. En el receso trae a tu repo lo nuevo de la U3: pídele ayuda a Claude Code con el README de unidad-3/taller/pedidos-web.

TALLER · 80 MIN · INDIVIDUAL

Módulo funcional y documentado

Sprint 2 de pedidos-web

Hoy: construir el **detalle de un pedido**, ordenar un ClientesService heredado y crear tu **primera Skill**.

Entregable

Detalle con tests (3+ commits) · ClientesService refactorizado, 4 tests en verde · Skill documentar-modulo · docs/modulos/ClientesService.md

1 · Detalle en 3 incrementos · 30′ | **2 · Refactor con red · 20′** | **3 · Skill documentar-modulo · 20′** | **4 · Cierre · 10′**

Descarga la guía del taller (PDF)

ffelipecuevasc.github.io/curso-ia-desarrolladores → Unidad 3

Licencia: Claude Team Standard · Stack: ASP.NET Core + Rider + xUnit

ESTA TARDE · DÍA 2

Unidad 4 · Análisis y diagnóstico de sistemas legados

✓ U1 ✓ U2 ✓ U3 · Módulo funcional y documentado

Hoy construiste el detalle del pedido en pasos chicos, refactorizaste ClientesService con red y creaste tu primera Skill. En la tarde: mirar sin tocar un sistema legado.

ffelipecuevasc.github.io/curso-ia-desarrolladores